Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 3

по курсу «Алгоритмы компьютерной графики»

Студент группы ИУ9-42Б Лавров Р. Д.

Преподаватель Цалкович П. А.

Москва 2025

1 Задача

Взять 2ю лабораторную и на ее основе заменить куб на фигуру по вариантам (в моём случае - сфера).

2 Основная теория

2.1 Инициализация графического окружения

Работа выполнена на базе второй лабораторной работы, в которой осуществлён переход от примитива в виде куба к более сложной трёхмерной фигуре — сфере. Для создания графического приложения используется библиотека GLFW, обеспечивающая создание окна размером 800x800 пикселей, управление контекстом OpenGL и обработку пользовательского ввода.

2.2 Математическая модель сферы

Сфера радиуса $R = 1$ строится путём задания поверхности набором четырёхугольных полос (quad strips). Для описания сферической поверхности используется параметрическое уравнение в сферических координатах:

$$\begin{cases} x = R\cos(\theta)\cos(\varphi) \\ y = R\sin(\theta) \\ z = R\cos(\theta)\sin(\varphi) \end{cases}$$

где $\theta \in [-\pi/2, \pi/2]$ — угол возвышения, $\varphi \in [0, 2\pi]$ — азимутальный угол.

В реализации сфера разбивается на 10 сегментов по широте. Для каждого сегмента вычисляются два последовательных угла θ_1 и θ_2, определяющие верхнее и нижнее основания пояса:

$$\theta_1 = -\frac{\pi}{2} + \frac{i}{n}\pi, \quad \theta_2 = -\frac{\pi}{2} + \frac{i+1}{n}\pi, \quad i = 0..n-1, \quad n = 10$$

Для каждого пояса определяются радиусы верхнего и нижнего оснований:

$$r_1 = R\cos(\theta_1), \quad r_2 = R\cos(\theta_2)$$

а также высоты:

$$y_1 = R\sin(\theta_1), \quad y_2 = R\sin(\theta_2)$$

2.3 Построение поверхности сферы

Для визуализации каждого пояса используется функция $\mathrm{draw_part}$, которая формирует четырёхугольную полосу (GL_QUAD_STRIP) с заданным количеством сторон (20 сегментов по долготе). Для каждого шага по долготе вычисляются координаты четырёх вершин, образующих грань:

$$\begin{aligned}
P_1 &= (r_1\cos\varphi,\ y_1,\ r_1\sin\varphi) \\
P_2 &= (r_2\cos\varphi,\ y_2,\ r_2\sin\varphi) \\
P_3 &= (r_1\cos(\varphi+\Delta\varphi),\ y_1,\ r_1\sin(\varphi+\Delta\varphi)) \\
P_4 &= (r_2\cos(\varphi+\Delta\varphi),\ y_2,\ r_2\sin(\varphi+\Delta\varphi))
\end{aligned}$$

где $\Delta\varphi = \frac{2\pi}{20}$ — шаг по долготе. Вершины соединяются в порядке $P_1 \to P_2 \to P_3 \to P_4$, формируя четырёхугольник, близкий к прямоугольнику на развёртке сферической поверхности.

2.4 Матрица трёхточечной проекции

В работе используется матрица трёхточечной проекции, обеспечивающая перспективное искажение с тремя точками схода:

$$\mathbf{M}_{\text{3P}} = \begin{bmatrix} 0.87 & -0.09 & 0.98 & 0.49 \\ 0 & 0.98 & 0.35 & 0.17 \\ 0.5 & 0.15 & -1.7 & -0.85 \\ 0 & 0 & 1 & 2 \end{bmatrix}$$

2.5 Композиция сцены и трансформации

Сцена содержит две сферы с различными наборами преобразований:

Вспомогательная сфера:

- Смещение: $\mathrm{glTranslatef}(0.6,\ 0.6,\ 0)$

- Масштабирование с коэффициентом 0.2 для уменьшения размера
- Применение матрицы проекции $\mathbf{M}_{\text{ЗР}}$

Основная сфера:

- Смещение: $\text{glTranslatef(-0.3, 0, 0)}$
- Масштабирование с переменным коэффициентом scale
- Применение матрицы проекции $\mathbf{M}_{\text{ЗР}}$
- Поворот вокруг оси X на угол θ_x и вокруг оси Y на угол θ_y

Порядок применения преобразований соответствует стандартному подходу OpenGL: результирующее преобразование имеет вид

$$\mathbf{P}_{\text{итог}} = \mathbf{T}_{\text{смещение}} \cdot \mathbf{S} \cdot \mathbf{M}_{\text{ЗР}} \cdot \mathbf{R}_x \cdot \mathbf{R}_y$$

где $\mathbf{S}$ — матрица масштабирования, $\mathbf{R}_x$ и $\mathbf{R}_y$ — матрицы поворота.

2.6 Интерактивное управление

Управление параметрами основной сферы осуществляется с клавиатуры через функцию обратного вызова key_callback:

- Клавиши-стрелки изменяют углы поворота θ_x и θ_y
- Клавиши ’+’ и ’-’ регулируют коэффициент масштабирования s
- Клавиша ’M’ переключает режим отображения между каркасным и твердотельным

3 Практическая реализация

Листинг 1: Основной код

```
import glfw
from OpenGL.GL import *
import math
```

```
4
5  angle_x, angle_y = 0, 0
6  wireframe = False
7  scale  = 1.0
8
9  def three_point_projection():
10     M_3P = [
11             0.87,  -0.09,  0.98,  0.49,
12             0,  0.98,  0.35,  0.17,
13             0.5,  0.15,  -1.7,  -0.85,
14             0,  0,  1,  2,
15         ]
16     return M_3P
17
18 def main():
19     if not glfw.init():
20         return
21     window = glfw.create_window(800, 800, "lab3", None, None)
22     if not window:
23         glfw.terminate()
24         return
25     glfw.make_context_current(window)
26     glfw.set_key_callback(window, key_callback)
27
28     glEnable(GL_DEPTH_TEST)
29     glClearColor(0.2,  0.2,  0.2,  1.0)
30
31     while not glfw.window_should_close(window):
32         display()
33         glfw.swap_buffers(window)
34         glfw.poll_events()
35
36     glfw.destroy_window(window)
37     glfw.terminate()
38
39 def draw_part(y, R1, R2, height, sides):
40     angle_step = 2*math.pi / sides
41     angle = 0
42     glBegin(GL_QUAD_STRIP)
43     for _ in range(sides+1):
44         glColor3f(0.5,  0.1,  0.1)
45         x1 = R1 * math.cos(angle)
46         z1 = R1 * math.sin(angle)
47         x2 = R2 * math.cos(angle)
48         z2 = R2 * math.sin(angle)
49         glVertex3f(x1,  y,  z1)
```

```
50            glVertex3f(x2, y+height, z2)
51            angle += angle_step
52        glEnd()
53
54
55    def draw_sphere():
56        R = 1
57        segments = 10
58        glPolygonMode(GL_FRONT_AND_BACK, GL_LINE if wireframe else GL_FILL)
59
60        for i in range(segments):
61            theta1 = -math.pi/2 + (i / segments) * math.pi
62            theta2 = -math.pi/2 + ((i + 1) / segments) * math.pi
63
64            y1 = R * math.sin(theta1)
65            y2 = R * math.sin(theta2)
66            r1 = R * math.cos(theta1)
67            r2 = R * math.cos(theta2)
68
69            height = y2 - y1
70            if r1 > 0 and r2 > 0:
71                draw_part(y1, r1, r2, height, 20)
72
73
74    def display():
75        global angle_x, angle_y, scale
76
77        glLoadIdentity()
78        glClear(GL_COLOR_BUFFER_BIT)
79        glClear(GL_DEPTH_BUFFER_BIT)
80        glMatrixMode(GL_PROJECTION)
81        glPushMatrix()
82
83        glTranslatef(0.6, 0.6, 0)
84        glScalef(0.2, 0.2, 0.2)
85        glMultMatrixf(three_point_projection())
86        draw_sphere()
87        glPopMatrix()
88
89        glTranslatef(-0.3, 0, 0)
90        glScalef(scale, scale, scale)
91        glMultMatrixf(three_point_projection())
92        glRotatef(angle_x, 1, 0, 0)
93        glRotatef(angle_y, 0, 1, 0)
94        draw_sphere()
95
```

```
96
97  def key_callback(window, key, scancode, action, mods):
98      global angle_x, angle_y, wireframe, scale
99      if action in [glfw.PRESS, glfw.REPEAT]:
100         if key == glfw.KEY_RIGHT:
101             angle_y -= 3
102         if key == glfw.KEY_LEFT:
103             angle_y += 3
104         if key == glfw.KEY_UP:
105             angle_x -= 3
106         if key == glfw.KEY_DOWN:
107             angle_x += 3
108         if key == glfw.KEY_KP_ADD or key == glfw.KEY_EQUAL:
109             scale += 0.1
110         if key == glfw.KEY_KP_SUBTRACT or key == glfw.KEY_MINUS:
111             scale = max(0.1, scale - 0.1)
112         if key == glfw.KEY_M:
113             wireframe = not wireframe
114
115 if __name__ == "__main__":
116     main()
```

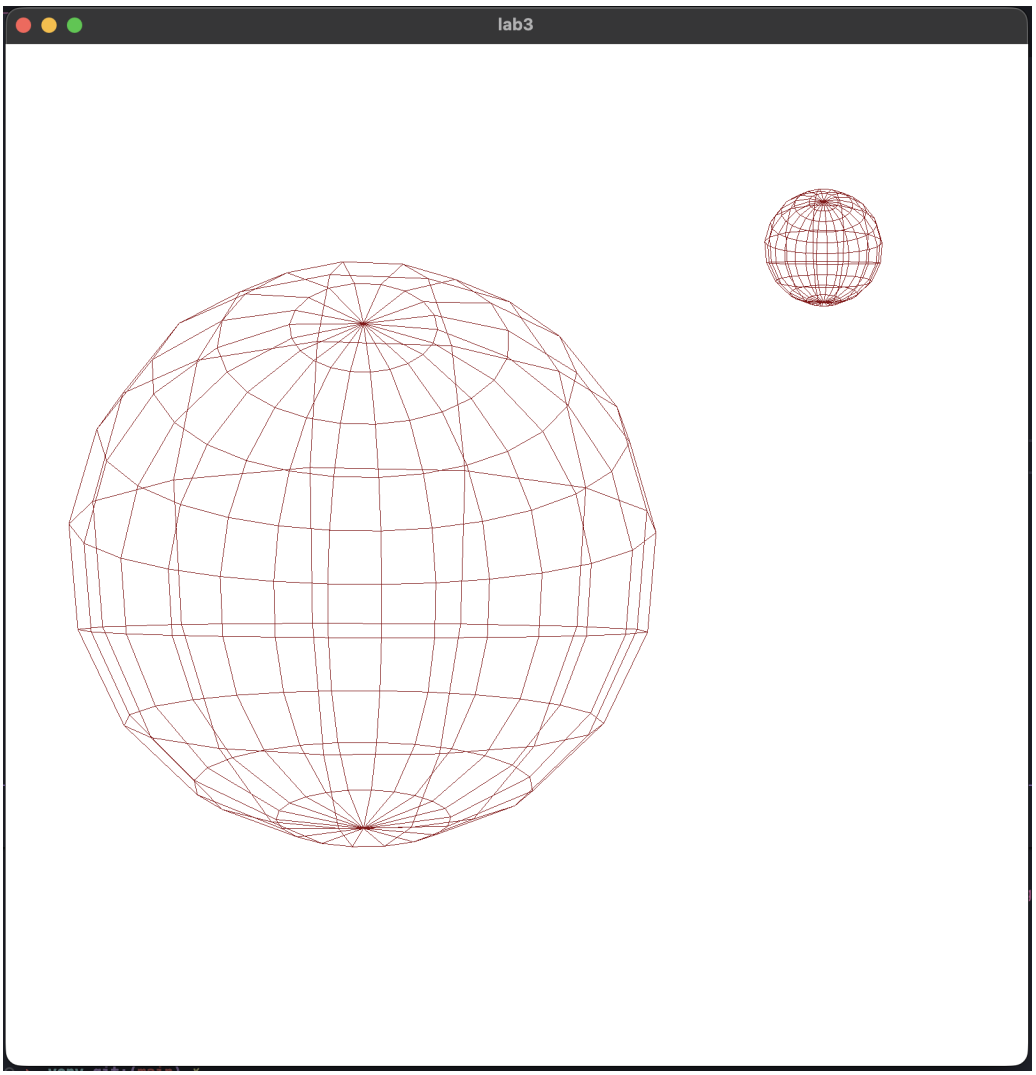


Рис. 1 — Результат работы

4 Вывод

В результате выполнения работы осуществлена модификация второй лабораторной работы: примитив в виде куба заменён на сферу, построенную путём аппроксимации поверхности с помощью параметрического уравнения. Реализован алгоритм генерации вершин сферы с разбиением по широте и долготе, обеспечивающий гладкую визуализацию криволинейной поверхности. Сохранены все функциональные возможности предыдущей работы: интерактивное управление поворотом и масштабированием, переключение режимов отображения, применение трёхточечной проекции.